Product and Category Management System

Relational Database Design & Implementation Guide

1. Overview

This document presents the database design for a Product and Category Management System. It defines the Category and Product entities, establishes their relationship through primary and foreign keys, and outlines the core operations required: inserting, updating, and deleting products, and generating category-wise product reports.

2. Entity Relationship Overview

The system is built around two core entities. A **Category** can contain many Products, while each **Product** belongs to exactly one Category. This is a classic **one-to-many** relationship, enforced using a foreign key.

CATEGORY
category_id (PK)
category_name
description

1 ■■■■< M

PRODUCT
product_id (PK)
category_id (FK)
product_name
price
stock_qty

3. Table Design

3.1 Category Table

Column	Data Type	Constraint	Description
category_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique category identifier
category_name	VARCHAR(100)	NOT NULL, UNIQUE	Name of the category
description	VARCHAR(255)	NULL	Optional category description
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation time

3.2 Product Table

Column	Data Type	Constraint	Description
product_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique product identifier
product_name	VARCHAR(150)	NOT NULL	Name of the product
category_id	INT	FOREIGN KEY -> Category(category_id)	Links product to its category

Column	Data Type	Constraint	Description
price	DECIMAL(10,2)	NOT NULL, CHECK (price >= 0)	Unit price of the product
stock_qty	INT	NOT NULL, DEFAULT 0	Available stock quantity
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation time

4. Primary and Foreign Key Relationships

category_id is the Primary Key of the Category table, uniquely identifying each category. It is referenced as a **Foreign Key** in the Product table, establishing the link between a product and the category it belongs to. This enforces referential integrity: a product cannot reference a category that does not exist, and (depending on the ON DELETE rule chosen) deleting a category can be restricted, cascaded, or set to NULL for its related products.

5. Table Creation (DDL)

```
CREATE TABLE Category (
    category_id     INT AUTO_INCREMENT PRIMARY KEY,
    category_name   VARCHAR(100) NOT NULL UNIQUE,
    description     VARCHAR(255),
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Product (
    product_id      INT AUTO_INCREMENT PRIMARY KEY,
    product_name    VARCHAR(150) NOT NULL,
    category_id     INT NOT NULL,
    price           DECIMAL(10,2) NOT NULL CHECK (price >= 0),
    stock_qty       INT NOT NULL DEFAULT 0,
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (category_id) REFERENCES Category(category_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);
```

6. Core Operations

6.1 Insert a New Product

```
INSERT INTO Product (product_name, category_id, price, stock_qty)
VALUES ('Wireless Mouse', 2, 799.00, 150);
```

6.2 Update an Existing Product

```
UPDATE Product
SET price = 749.00, stock_qty = stock_qty + 50
WHERE product_id = 101;
```

6.3 Delete a Product

```
DELETE FROM Product
WHERE product_id = 101;
```

Note: Because Product.category_id references Category.category_id with ON DELETE RESTRICT, a Category cannot be deleted while Products still reference it — those Products must be reassigned or removed first.

7. Category-wise Product Report

A category-wise report joins the two tables and aggregates product data per category, typically showing total products, total stock value, and average price.

```
SELECT
    c.category_name,
    COUNT(p.product_id)        AS total_products,
    SUM(p.stock_qty)           AS total_stock,
    ROUND(AVG(p.price), 2)     AS avg_price,
    SUM(p.price * p.stock_qty) AS total_stock_value
FROM Category c
LEFT JOIN Product p ON c.category_id = p.category_id
GROUP BY c.category_id, c.category_name
ORDER BY c.category_name;
```

Sample Report Output

Category	Total Products	Total Stock	Avg. Price	Stock Value
Electronics	18	540	1,249.50	6,74,730.00
Groceries	42	1,820	89.75	1,63,345.00
Furniture	9	76	5,499.00	4,17,924.00

8. Suggested Next Topic

A natural extension of this project is an **Order and Inventory Management System**, which would add Customer and Order entities, link Orders to Products through an OrderDetails table (many-to-many),

and automatically deduct stock when an order is placed. This introduces transaction handling, multi-table joins, and stock-level triggers — a good next step after mastering the Product and Category design.